

vLLM tuning cheat sheet

Six settings, what each one is really a knob on, and which way to move it. Every number here is **vLLM v0.27.1**. Defaults move between releases — check **vllm --version** first.

THE SIX SETTINGS

SETTING	DEFAULT	A KNOB ON	WHICH WAY TO MOVE IT
--max-num-seqs	1024 / 256	A ceiling on the batch, not the batch you get.	Lower it to shorten the queue, and pay in throughput. Raising it usually does nothing, because capacity binds first — except at short contexts on a 256-default card.
--gpu-memory-utilization	0.92	The share of the card vLLM will touch at all. Weights, activation peak and graph buffers come out of <i>inside</i> it.	0.95 is the cheapest capacity available, though nothing like the largest. You court fragmentation or a co-tenant, not a long request: one that cannot get blocks is preempted, not crashed.
--max-num-batched-tokens	16,384 / 8,192 8,192 / 2,048	The per-step budget. Requests already generating come off the top, one token each.	Smaller protects the per-token speed of those already generating, at the newcomer's first token. Larger does the reverse. No setting is correct. Floor: at least --max-num-seqs .
--enable-prefix-caching	on	Reuse, exact-match from the first token.	You do not turn this on. Check nobody turned it off, and check the hit rate: a warm cache and a cold one measure two different products.
--kv-cache-dtype fp8	off	Cache bytes. Halving them roughly doubles the conversations that fit.	Take it for the capacity, then two checks: read the backend line, because one without FP8 support falls back quietly; and re-run your own eval, because the published accuracy evidence covers weights and activations, never the cache.
--enforce-eager	off	Compile and CUDA graph capture. Fast start, slower decode.	Development only. One exception: where capacity binds, the graph buffers it skips go back to the KV cache, so measure both ways.

THE TWO DEFAULTS THAT DEPEND ON YOUR CARD — V0.27.1

CARD	--max-num-seqs	BATCHED TOKENS, OFFLINE	...THROUGH THE API SERVER
B200	1024	16,384	8,192
H100	1024	16,384	8,192
H200	1024	16,384	8,192
A100	256	8,192	2,048
L40S	256	8,192	2,048

Two tiers, not five cards: device memory, with one exception. Under 70 GiB you get the smaller default, and the A100 gets it too, by name. Most people are serving, so most have the highlighted number.

WHAT TO MEASURE

1. **Time to first token** (TTFT) — prompt reading and queueing.
2. **Time per output token** (TPOT) — memory bandwidth and batch size.
3. **Throughput** — the aggregate.
4. **Goodput** — throughput of requests that met your target. The only one that maps to money.

Which wall are you against? Put **vllm:num_requests_running** against part one's ridge point for your card — 296 on an H100, 153 on an A100, 419 on an L40S — and watch **vllm:num_preemptions_total**.

HOW TO SWEEP IT YOURSELF

1. **Pin your request shapes first.** Prompt lengths, output lengths, and how much your prompts share a prefix. If you cannot state those three, that is the task, not the tuning.
2. **Then one setting at a time**, in this order: **--gpu-memory-utilization**, **--max-num-seqs**, **--max-num-batched-tokens**.
3. **Re-read the KV cache line** in the startup log afterwards: the token budget sets the activation peak, and that peak comes out of the fence you set first.
4. **Record p99 next to the median.** A setting that improves the median and wrecks the tail made your product worse.
5. **Expect most of it to do nothing**, and write down what did not move, or you will sweep it again in six months.

Preemptions climbing under load you used to handle means capacity is binding, and the fence and the 8-bit cache are where the room is. Running far under the ridge with none means capacity is fine and you are losing latency elsewhere, so the ceiling and the token budget are the pair to move.

Sources: the per-card numbers are **get_batch_defaults** in **vllm/engine/arg_utils.py**; the 0.92 fence is the field default in **vllm/config/cache.py**; both read at tag **v0.27.1**. From *Tuning vLLM: What Every Setting Does to the Arithmetic*, part five of *LLM Inference, Measured*, by Satsawat Natakarnkitkul.